汇编语言第二次作业

作业题目 2：按要求打印 ASCII 表

基本要求：输出 ASCII 表中的小写字母部分，要求每行 13 个字符。

1)→用 loop 指令实现；

注意双重循环中 CX 值的保存和恢复；



2)→用条件跳转指令实现；

3)→用 C 语言实现后察看反汇编代码并加注释；

在上次搭建好的 DOSBox + MASM 汇编环境上继续做，不需要重新安装软件。一共要做三种实现：

1. 用 `LOOP` 指令实现：每行输出 13 个小写字母，注意双重循环中 `CX` 的保存和恢复。
2. 用条件跳转指令实现：同样输出 `a` 到 `z`，每行 13 个字符。
3. 用 C 语言实现：运行后查看对应的反汇编代码，并添加注释。

我们直接在原来的 `D:\myassembly` 文件夹里继续做。

第一部分：用 `LOOP` 指令实现

第 1 步：新建汇编文件

文件名保存为 `ascii_loop.asm`，保存位置为：

代码块

```
D:\myassembly\ascii_loop.asm
```

代码块

```
.model small
.stack 100h
.code
START:
```

```
    mov bl, 'a'
    mov cx, 2
L1:
    push cx
    mov cx, 13
L2:
    mov dl, bl
    mov ah, 02h
    int 21h
    mov dl, ' '
    mov ah, 02h
    int 21h
    inc bl
    loop L2
    mov dl, 0Dh
    mov ah, 02h
    int 21h
    mov dl, 0Ah
    mov ah, 02h
    int 21h
    pop cx
    loop L1
    mov ax, 4C00h
    int 21h
END START
```

第 2 步：在 DOSBox 中编译运行

打开你之前配置好的 DOSBox，依次输入：

代码块

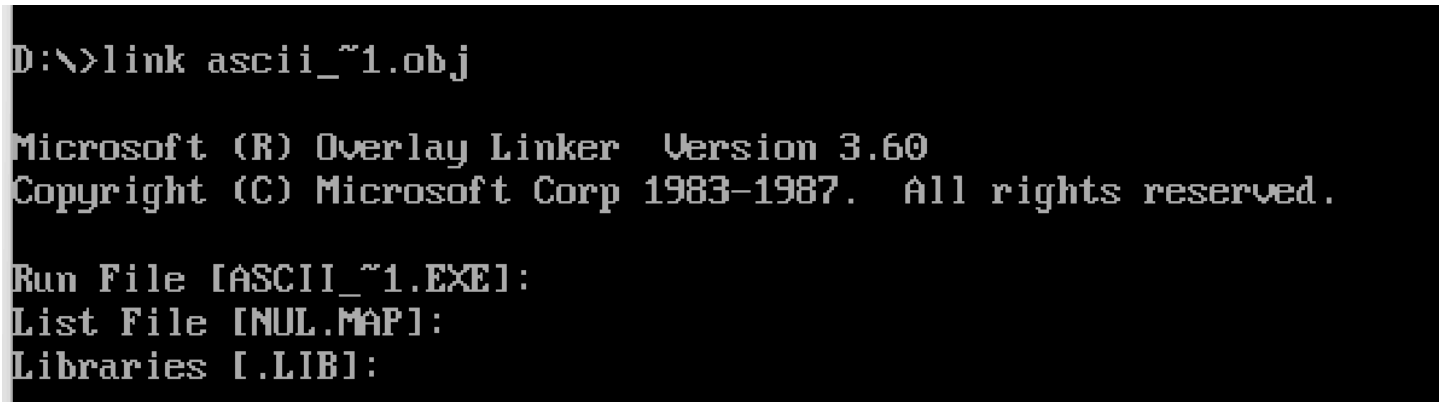
```
masm ascii_~1.asm
```

如果出现目标文件名、列表文件等提示，按回车接受默认值即可。

然后链接：

代码块

```
link ascii_~1.obj
```

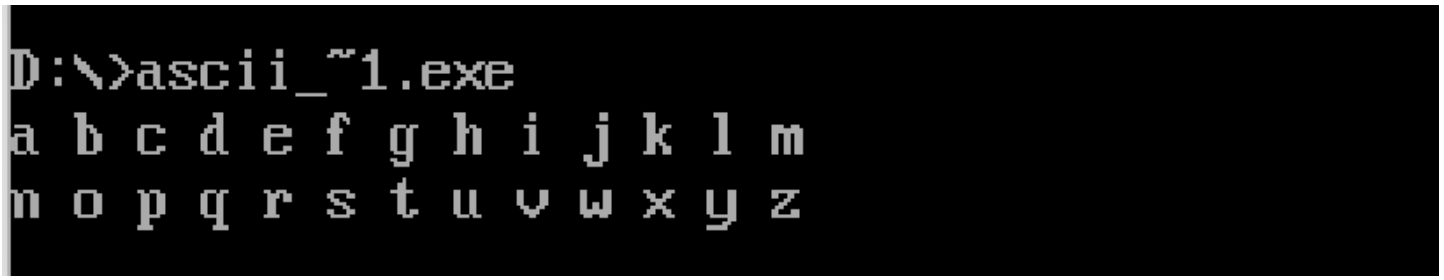


同样按回车接受默认选项。

最后运行：

```
ascii_~1.exe
```

输出如下：



第 3 步：理解双重循环中的 `CX`

这是本题最重要的知识点。

外层循环CX = 2控制总共输出两行。每行输出结束后，外层循环的 CX 才减 1。

内层循环CX = 13控制每行输出 13 个字母。每执行一次 `LOOP L2`，CX 就减 1。

为什么使用 PUSH 和 POP？因为内层循环会把 CX 减到 0。如果不先保存外层 CX，外层循环的次数也会丢失。

第二部分：用条件跳转指令实现

这次不使用 `LOOP`，改用 `CMP`、`JE`、`DEC`、`JMP` 等指令控制循环。

第 1 步：新建文件

保存为：

```
```

```
D:\myassembly\jump.asm
```

代码如下：

代码块

```
.model small
.stack 100h

.code
START:
    mov bl, 'a'
    mov di, 2

L1:
    mov si, 13

L2:
    cmp si, 0
    je NEXT_LINE

    mov dl, bl
    mov ah, 02h
    int 21h

    mov dl, ' '
    mov ah, 02h
    int 21h

    inc bl
    dec si
    jmp L2

NEXT_LINE:
    mov dl, 0Dh
    mov ah, 02h
    int 21h
    mov dl, 0Ah
    mov ah, 02h
    int 21h

    dec di
    jnz L1

    mov ax, 4C00h
    int 21h
```

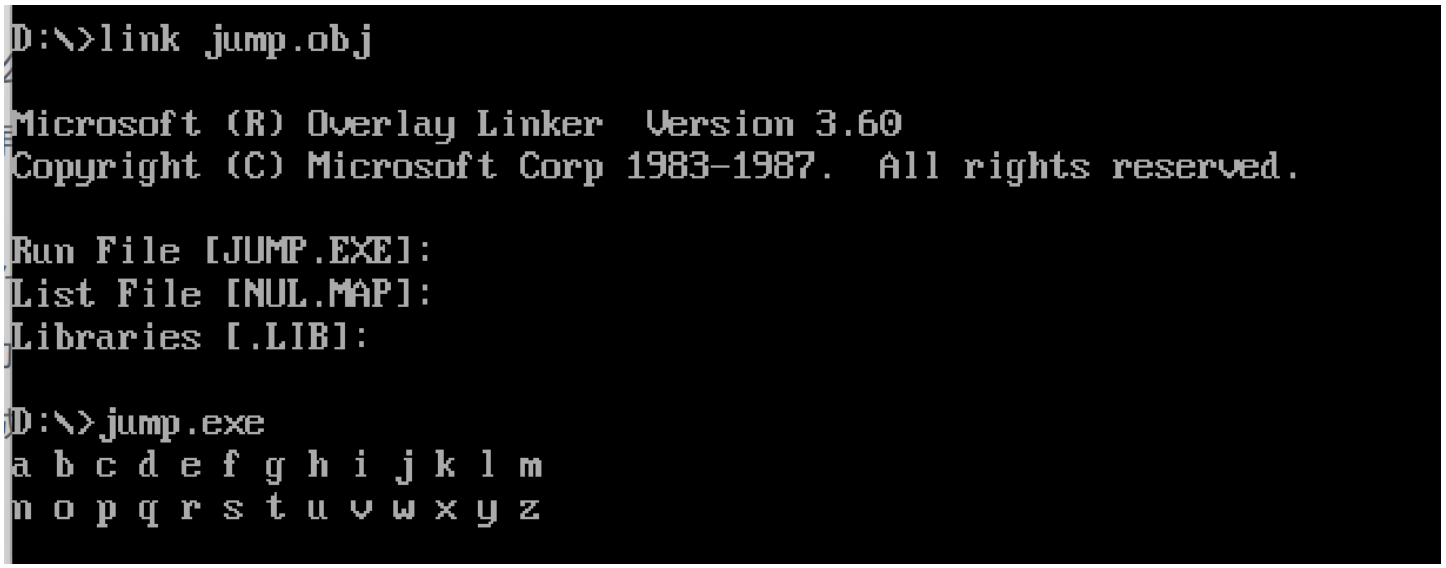
```
41
42  END START
```

第 2 步：编译运行

代码块

```
1  masm jump.asm
2  link jump.obj
3  jump.exe
```

输出应该与第一部分一致。



第 3 步：这两种方法有什么区别？

比较内容	LOOP 方法	条件跳转方法
循环控制	CX 自动减 1	手动 DEC 计数
判断方式	CX 是否为 0	CMP、JE、JNZ 等
跳转指令	LOOP	条件跳转和 JMP
本题的重点	保存、恢复 CX	明确控制循环条件

第二种方法使用 `SI` 控制每行字母数、 `DI` 控制行数，不需要像第一种方法那样保存和恢复外层 `CX` 。

第三部分：用 C 语言实现，再查看反汇编代码并添加注释

这部分和前两部分不同：先写 C 程序，再让编译器把它转换成汇编代码，观察高级语言和汇编指令之间的关系。

第 1 步：编写 C 语言程序

新建文件 `ascii.c`，保存到 `D:\myassembly`。

代码块

```
#include <stdio.h>

int main(void)
{
    char ch;

    // 遍历所有小写英文字母
    for (ch = 'a'; ch <= 'z'; ch++)
    {
        printf("%c ", ch);

        // 每输出 13 个字母就换行
        if ((ch - 'a' + 1) % 13 == 0)
        {
            printf("\n");
        }
    }

    return 0;
}
```

这段程序的输出与前两种汇编实现相同。

第 2 步：编译并生成汇编代码

bat

代码块

```
cd /d D:\myassembly
```

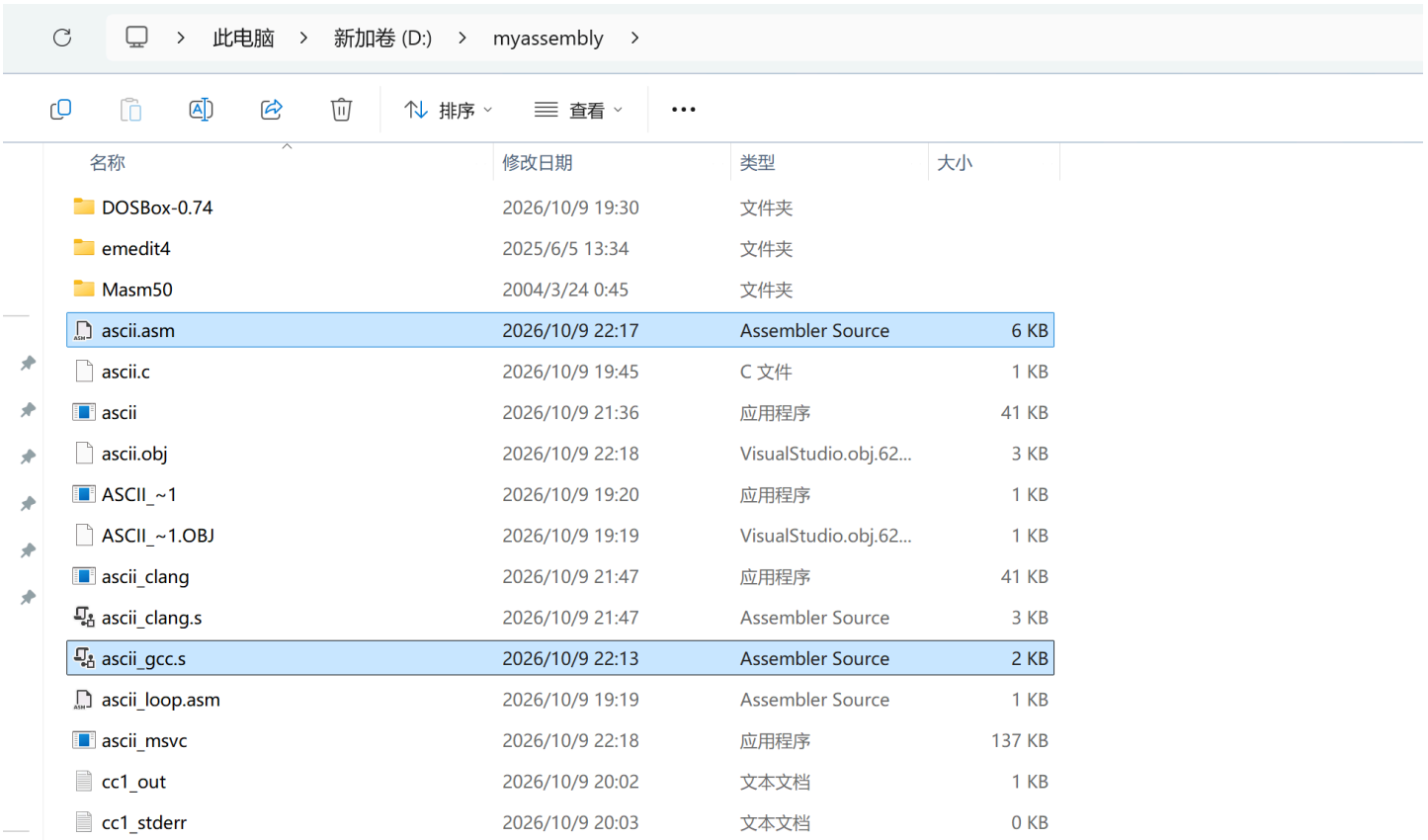
编译并生成包含 C 源码对应信息的汇编列表：

Bash

代码块

```
gcc -S -masm=intel ascii.c -o ascii_gcc.s
```

生成的 `ascii_gcc.s` 用 VS Code 打开查看。



第 3 步：查看反汇编并添加注释

打开生成的汇编文件看到编译器产生的指令。具体指令会因编译器、优化等级和目标架构而不同，不一定与前两部分的手写汇编相同。

重点找出与以下 C 代码对应的逻辑：

C 语言操作	汇编层面可能出现的操作	含义
ch = 'a'	mov	初始化字符变量
ch <= 'z'	cmp、条件跳转	判断是否到达字母表末尾
ch++	inc 或 add	转到下一个字母
% 13 == 0	比较、算术运算和条件跳转	判断是否需要换行
printf()	函数调用指令，如 call	调用输出函数

在生成的汇编代码旁边添加注释，解释循环判断、字符递增、条件跳转和函数调用的作用。

```
.file   "ascii.c"                   # 源文件名为 ascii.c
.intel_syntax noprefix              # 使用 Intel 汇编语法
.text                               # 代码段

.section .rdata,"dr"                # 定义只读数据段
.LC0:
    .ascii "%c \0"                  # printf 的格式字符串：输出一个字符和一个空格，\0 表示字符串结束

.text                               # 切换回代码段
.globl "main"                       # 声明 main 为全局符号
.def "main"; .scl 2; .type 32; .endef  # 定义 main 函数的符号信息

.seh_proc "main"                    # 开始记录 main 函数的异常处理信息
"main":
    push rbp                        # 保存旧的栈帧指针
    .seh_pushreg rbp                # 记录保存 rbp 的操作
    mov rbp, rsp                    # 建立当前函数的栈帧
    .seh_setframe rbp, 0            # 记录栈帧基址
    sub rsp, 48                     # 在栈上分配 48 字节空间
    .seh_stackalloc 48              # 记录栈空间分配信息
    .seh_endprologue                # 函数序言结束

    call "__main"                   # 调用运行时初始化函数

    mov BYTE PTR -1[rbp], 97        # 将 97（字符 'a' 的 ASCII 码）存入局部变量
    jmp .L2                         # 跳转到循环条件判断处

.L4:                                # 循环体开始
    movsx eax, BYTE PTR -1[rbp]     # 将当前字符取出并进行符号扩展，保存到 eax
    lea rcx, .LC0[rip]              # 将格式字符串 "%c " 的地址放入 rcx（第一个参数）
    mov edx, eax                    # 将当前字符放入 edx（第二个参数）
    call "printf"                   # 调用 printf 输出当前字符和空格

    movsx eax, BYTE PTR -1[rbp]     # 再次读取当前字符，并进行符号扩展
    lea ecx, -96[rax]               # 计算当前字符对应的数值：字符 ASCII 码减去 96
                                    # 例如 'a' 为 97，97 - 96 = 1
    movsxd rax, ecx                 # 将 ecx 中的有符号 32 位整数扩展为 64 位整数
    imul rax, rax, 1321528399       # 乘以编译器生成的常数，用于优化除法计算
    shr rax, 32                     # 逻辑右移 32 位，提取乘法结果的高 32 位
    mov edx, eax                    # 将计算结果复制到 edx
    sar edx, 2                      # 算术右移 2 位，相当于有符号数除以 4 的近似步骤
    mov eax, ecx                    # 将原数值复制到 eax
    sar eax, 31                     # 根据 eax 的符号位生成符号调整值
    sub edx, eax                    # 修正除法结果的符号
    mov eax, edx                    # 将计算结果复制到 eax
    add eax, eax                    # eax = eax * 2
    add eax, edx                    # eax = eax * 3
    sal eax, 2                      # eax 左移 2 位，相当于 eax = eax * 4
    add eax, edx                    # 得到 eax = 原商 * 13
    sub ecx, eax                    # 计算余数：原数值减去商乘以 13
```